

旁注 在连接中 EOF 意味什么?

EOF 的概念常常使人们感到迷惑,尤其是在因特网连接的上下文中。首先,我们需要理解其实并没有像 EOF 字符这样的一个东西。进一步来说,EOF 是由内核检测到的一种条件。应用程序在它接收到一个由 read 函数返回的零返回码时,它就会发现出 EOF 条件。对于磁盘文件,当前文件位置超出文件长度时,会发生 EOF。对于因特网连接,当一个进程关闭连接它的那一端时,会发生 EOF。连接另一端的进程在试图读取流中最后一个字节之后的字节时,会检测到 EOF。

11.5 Web 服务器

迄今为止,我们已经在简单的 echo 服务器的上下文中讨论了网络编程。在这一节里,我们将向你展示如何利用网络编程的基本概念,来创建你自己的虽小但功能齐全的 Web 服务器。

11.5.1 Web 基础

Web 客户端和服务端之间的交互用的是一个基于文本的应用级协议,叫做 HTTP (Hypertext Transfer Protocol, 超文本传输协议)。HTTP 是一个简单的协议。一个 Web 客户端(即浏览器)打开一个到服务器的因特网连接,并且请求某些内容。服务器响应所请求的内容,然后关闭连接。浏览器读取这些内容,并把它显示在屏幕上。

Web 服务和常规的文件检索服务(例如 FTP)有什么区别呢?主要的区别是 Web 内容可以用一种叫做 HTML(Hypertext Markup Language, 超文本标记语言)的语言来编写。一个 HTML 程序(页)包含指令(标记),它们告诉浏览器如何显示这页中的各种文本和图形对象。例如,代码

```
<b> Make me bold! </b>
```

告诉浏览器用粗体字类型输出 `<b>` 和 `</b>` 标记之间的文本。然而,HTML 真正的强大之处在于一个页面可以包含指针(超链接),这些指针可以指向存放在任何因特网主机上的内容。例如,一个格式如下的 HTML 行

```
<a href="http://www.cmu.edu/index.html">Carnegie Mellon</a>
```

告诉浏览器高亮显示文本对象“Carnegie Mellon”,并且创建一个超链接,它指向存放在 CMU Web 服务器上叫做 index.html 的 HTML 文件。如果用户单击了这个高亮文本对象,浏览器就会从 CMU 服务器中请求相应的 HTML 文件并显示它。

旁注 万维网的起源

万维网是 Tim Berners-Lee 发明的,他是一位在瑞典物理实验室 CERN(欧洲粒子物理研究所)工作的软件工程师。1989 年, Berners-Lee 写了一个内部备忘录,提出了一个分布式超文本系统,它能连接“用链接组成的笔记的网(web of notes with links)”。提出这个系统的目的是帮助 CERN 的科学家共享和管理信息。在接下来的两年多里, Berners-Lee 实现了第一个 Web 服务器和 Web 浏览器之后,在 CERN 内部以及其他一些网站中, Web 发展出了小规模拥护者。1993 年一个关键事件发生了, Marc Andreessen(他后来创建了 Netscape)和他在 NCSA 的同事发布了一种图形化的浏览器,叫做 MOSAIC,可以在三种主要的平台上所使用: Unix、Windows 和 Macintosh。在 MOSAIC 发布后,对 Web 的兴趣爆发了, Web 网站以每年 10 倍或更高的数量增长。到 2015 年,世界上已经有超过 975 000 000 个 Web 网站了(源自 Netcraft Web Survey)。

11.5.2 Web 内容

对于 Web 客户端和服务端而言，内容是与一个 MIME (Multipurpose Internet Mail Extensions，多用途的网际邮件扩充协议) 类型相关的字节序列。图 11-23 展示了一些常用的 MIME 类型。

MIME 类型	描述
text/html	HTML 页面
text/plain	无格式文本
application/postscript	Postscript 文档
image/gif	GIF 格式编码的二进制图像
image/png	PNG 格式编码的二进制图像
image/jpeg	JPEG 格式编码的二进制图像

图 11-23 MIME 类型示例

Web 服务器以两种不同的方式向客户端提供内容：

- 取一个磁盘文件，并将它的内容返回给客户端。磁盘文件称为静态内容 (static content)，而返回文件给客户端的过程称为服务静态内容 (serving static content)。
- 运行一个可执行文件，并将它的输出返回给客户端。运行时可执行文件产生的输出称为动态内容 (dynamic content)，而运行程序并返回它的输出到客户端的过程称为服务动态内容 (serving dynamic content)。

每条由 Web 服务器返回的内容都是和它管理的某个文件相关联的。这些文件中的每一个都有一个唯一的名字，叫做 URL (Universal Resource Locator，通用资源定位符)。例如，URL

`http://www.google.com:80/index.html`

表示因特网主机 `www.google.com` 上一个称为 `/index.html` 的 HTML 文件，它是由一个监听端口 80 的 Web 服务器管理的。端口号是可选的，默认为知名的 HTTP 端口 80。可执行文件的 URL 可以在文件名后包括程序参数。“?” 字符分隔文件名和参数，而且每个参数都用 “&” 字符分隔开。例如，URL

`http://bluefish.ics.cs.cmu.edu:8000/cgi-bin/adder?15000&213`

标识了一个叫做 `/cgi-bin/adder` 的可执行文件，会带两个参数字符串 15000 和 213 来调用它。在事务过程中，客户端和服务端使用的是 URL 的不同部分。例如，客户端使用前缀

`http://www.google.com:80`

来决定与哪类服务器联系，服务器在哪里，以及它监听的端口号是多少。服务器使用后缀 `/index.html`

来发现在它文件系统中的文件，并确定请求的是静态内容还是动态内容。

关于服务器如何解释一个 URL 的后缀，有几点需要理解：

- 确定一个 URL 指向的是静态内容还是动态内容没有标准的规则。每个服务器对它所管理的文件都有自己的规则。一种经典的 (老式的) 方法是，确定一组目录，例如 `cgi-bin`，所有的可执行性文件都必须存放这些目录中。
- 后缀中的最开始的那个 “/” 不表示 Linux 的根目录。相反，它表示的是被请求内容类型的主目录。例如，可以将一个服务器配置成这样：所有的静态内容存放在目录 `/usr/httpd/html` 下，而所有的动态内容都存放在目录 `/usr/httpd/cgi-bin` 下。

- 最小的 URL 后缀是 “/” 字符，所有服务器将其扩展为某个默认的主页，例如 /index.html。这解释了为什么简单地在浏览器中键入一个域名就可以取出一个网站的主页。浏览器在 URL 后添加缺失的 “/”，并将之传递给服务器，服务器又把 “/” 扩展到某个默认的文件名。

11.5.3 HTTP 事务

因为 HTTP 是基于在因特网连接上传送的文本行的，我们可以使用 Linux 的 TELNET 程序来和因特网上的任何 Web 服务器执行事务。对于调试在连接上通过文本行来与客户端对话的服务器来说，TELNET 程序是非常便利的。例如，图 11-24 使用 TELNET 向 AOL Web 服务器请求主页。

1	linux> telnet www.aol.com 80	Client: open connection to server
2	Trying 205.188.146.23...	Telnet prints 3 lines to the terminal
3	Connected to aol.com.	
4	Escape character is '^['.	
5	GET / HTTP/1.1	Client: request line
6	Host: www.aol.com	Client: required HTTP/1.1 header
7		Client: empty line terminates headers
8	HTTP/1.0 200 OK	Server: response line
9	MIME-Version: 1.0	Server: followed by five response headers
10	Date: Mon, 8 Jan 2010 4:59:42 GMT	
11	Server: Apache-Coyote/1.1	
12	Content-Type: text/html	Server: expect HTML in the response body
13	Content-Length: 42092	Server: expect 42,092 bytes in the response body
14		Server: empty line terminates response headers
15	<html>	Server: first HTML line in response body
16	...	Server: 766 lines of HTML not shown
17	</html>	Server: last HTML line in response body
18	Connection closed by foreign host.	Server: closes connection
19	linux>	Client: closes connection and terminates

图 11-24 一个服务静态内容的 HTTP 事务

在第 1 行，我们从 Linux shell 运行 TELNET，要求它打开一个到 AOL Web 服务器的连接。TELNET 向终端打印三行输出，打开连接，然后等待我们输入文本(第 5 行)。每次输入一个文本行，并键入回车键，TELNET 会读取该行，在后面加上回车和换行符号(在 C 的表示中为 “\r\n”)，并且将这一行发送到服务器。这是和 HTTP 标准相符的，HTTP 标准要求每个文本行都由一对回车和换行符来结束。为了发起事务，我们输入一个 HTTP 请求(第 5~7 行)。服务器返回 HTTP 响应(第 8~17 行)，然后关闭连接(第 18 行)。

1. HTTP 请求

一个 HTTP 请求的组成是这样的：一个请求行(request line)(第 5 行)，后面跟随零个或多个请求报头(request header)(第 6 行)，再跟随一个空的文本行来终止报头列表(第 7 行)。一个请求行的形式是

method URI version

HTTP 支持许多不同的方法，包括 GET、POST、OPTIONS、HEAD、PUT、DELETE 和 TRACE。我们将只讨论广为应用的 GET 方法，大多数 HTTP 请求都是这种类型的。

GET 方法指导服务器生成和返回 URI(Uniform Resource Identifier, 统一资源标识符)标识的内容。URI 是相应的 URL 的后缀, 包括文件名和可选的参数。[⊖]

请求行中的 version 字段表明了该请求遵循的 HTTP 版本。最新的 HTTP 版本是 HTTP/1.1 [37]。HTTP/1.0 是从 1996 年沿用至今的老版本 [6]。HTTP/1.1 定义了一些附加的报头, 为诸如缓冲和安全等高级特性提供支持, 它还支持一种机制, 允许客户端和服务器的同一条持久连接(persistent connection)上执行多个事务。在实际中, 两个版本是互相兼容的, 因为 HTTP/1.0 的客户端和服务器的会简单地忽略 HTTP/1.1 的报头。

总的来说, 第 5 行的请求行要求服务器取出并返回 HTML 文件/index.html。它也告知服务器请求剩下的部分是 HTTP/1.1 格式的。

请求报头为服务器提供了额外的信息, 例如浏览器的商标名, 或者浏览器理解的 MIME 类型。请求报头的格式为

header-name: header-data

针对我们的目的, 唯一需要关注的报头是 Host 报头(第 6 行), 这个报头在 HTTP/1.1 请求中是需要的, 而在 HTTP/1.0 请求中是不需要的。代理缓存(proxy cache)会使用 Host 报头, 这个代理缓存有时作为浏览器和管理被请求文件的原始服务器(origin server)的中介。客户端和原始服务器之间, 可以有多个代理, 即所谓的代理链(proxy chain)。Host 报头中的数据指示了原始服务器的域名, 使得代理链中的代理能够判断它是否可以在本地缓存中拥有一个被请求内容的副本。

继续图 11-24 中的示例, 第 7 行的空文本行(通过在键盘上键入回车键生成的)终止了报头, 并指示服务器发送被请求的 HTML 文件。

2. HTTP 响应

HTTP 响应和 HTTP 请求是相似的。一个 HTTP 响应的组成是这样的: 一个响应行(response line)(第 8 行), 后面跟随着零个或更多的响应报头(response header)(第 9~13 行), 再跟随一个终止报头的空行(第 14 行), 再跟随一个响应主体(response body)(第 15~17 行)。一个响应行的格式是

version status-code status-message

version 字段描述的是响应所遵循的 HTTP 版本。状态码(status-code)是一个 3 位的正整数, 指明对请求的处理。状态消息(status message)给出与错误代码等价的英文描述。图 11-25 列出了一些常见的状态码, 以及它们相应的消息。

状态代码	状态消息	描述
200	成功	处理请求无误
301	永久移动	内容已移动到location头中指定的主机上
400	错误请求	服务器不能理解请求
403	禁止	服务器无权访问所请求的文件
404	未发现	服务器不能找到所请求的文件
501	未实现	服务器不支持请求的方法
505	HTTP 版本不支持	服务器不支持请求的版本

图 11-25 一些 HTTP 状态码

⊖ 实际上, 只有当浏览器请求内容时, 这才是真的。如果代理服务器请求内容, 那么这个 URI 必须是完整的 URL。

第 9~13 行的响应报头提供了关于响应的附加信息。针对我们的目的，两个最重要的报头是 Content-Type(第 12 行)，它告诉客户端响应主体中内容的 MIME 类型；以及 Content-Length(第 13 行)，用来指示响应主体的字节大小。

第 14 行的终止响应报头的空文本行，其后跟随着响应主体，响应主体中包含着被请求的内容。

11.5.4 服务动态内容

如果我们停下来考虑一下，一个服务器是如何向客户端提供动态内容的，就会发现一些问题。例如，客户端如何将程序参数传递给服务器？服务器如何将这些参数传递给它所创建的子进程？服务器如何将子进程生成内容所需要的其他信息传递给子进程？子进程将它的输出发送到哪里？一个称为 CGI(Common Gateway Interface, 通用网关接口)的实际标准的出现解决了这些问题。

1. 客户端如何将程序参数传递给服务器

GET 请求的参数在 URI 中传递。正如我们看到的，一个“?”字符分隔了文件名和参数，而每个参数都用一个“&”字符分隔开。参数中不允许有空格，而必须用字符串“%20”来表示。对其他特殊字符，也存在着相似的编码。

旁注 在 HTTP POST 请求中传递参数

HTTP POST 请求的参数是在请求主体中而不是 URI 中传递的。

2. 服务器如何将参数传递给子进程

在服务器接收一个如下的请求后

```
GET /cgi-bin/adder?15000&213 HTTP/1.1
```

它调用 fork 来创建一个子进程，并调用 execve 在子进程的上下文中执行 /cgi-bin/adder 程序。像 adder 这样的程序，常常被称为 CGI 程序，因为它们遵守 CGI 标准的规则。而且，因为许多 CGI 程序是用 Perl 脚本编写的，所以 CGI 程序也常被称为 CGI 脚本。在调用 execve 之前，子进程将 CGI 环境变量 QUERY_STRING 设置为“15000&213”，adder 程序在运行时可以用 Linux getenv 函数来引用它。

3. 服务器如何将其他信息传递给子进程

CGI 定义了大量的其他环境变量，一个 CGI 程序在它运行时可以设置这些环境变量。图 11-26 给出了其中的一部分。

环境变量	描述
QUERY_STRING	程序参数
SERVER_PORT	父进程侦听的端口
REQUEST_METHOD	GET 或 POST
REMOTE_HOST	客户端的域名
REMOTE_ADDR	客户端的点分十进制 IP 地址
CONTENT_TYPE	只对 POST 而言：请求体的 MIME 类型
CONTENT_LENGTH	只对 POST 而言：请求体的字节大小

图 11-26 CGI 环境变量示例

4. 子进程将它的输出发送到哪里

一个 CGI 程序将它的动态内容发送到标准输出。在子进程加载并运行 CGI 程序之前，

它使用 Linux `dup2` 函数将标准输出重定向到和客户端相关联的已连接描述符。因此，任何 CGI 程序写到标准输出的东西都会直接到达客户端。

注意，因为父进程不知道子进程生成的内容的类型或大小，所以子进程就要负责生成 `Content-type` 和 `Content-length` 响应报头，以及终止报头的空行。

图 11-27 展示了一个简单的 CGI 程序，它对两个参数求和，并返回带结果的 HTML 文件给客户端。图 11-28 展示了一个 HTTP 事务，它根据 `adder` 程序提供动态内容。

```

1  #include "csapp.h"
2
3  int main(void) {
4      char *buf, *p;
5      char arg1[MAXLINE], arg2[MAXLINE], content[MAXLINE];
6      int n1=0, n2=0;
7
8      /* Extract the two arguments */
9      if ((buf = getenv("QUERY_STRING")) != NULL) {
10         p = strchr(buf, '&');
11         *p = '\0';
12         strcpy(arg1, buf);
13         strcpy(arg2, p+1);
14         n1 = atoi(arg1);
15         n2 = atoi(arg2);
16     }
17
18     /* Make the response body */
19     sprintf(content, "QUERY_STRING=%s", buf);
20     sprintf(content, "Welcome to add.com: ");
21     sprintf(content, "%sTHE Internet addition portal.\r\n<p>", content);
22     sprintf(content, "%sThe answer is: %d + %d = %d\r\n<p>",
23             content, n1, n2, n1 + n2);
24     sprintf(content, "%sThanks for visiting!\r\n", content);
25
26     /* Generate the HTTP response */
27     printf("Connection: close\r\n");
28     printf("Content-length: %d\r\n", (int)strlen(content));
29     printf("Content-type: text/html\r\n\r\n");
30     printf("%s", content);
31     fflush(stdout);
32
33     exit(0);
34 }

```

图 11-27 对两个整数求和的 CGI 程序


```

1  linux> telnet kittyhawk.cmcl.cs.cmu.edu 8000  Client: open connection
2  Trying 128.2.194.242...
3  Connected to kittyhawk.cmcl.cs.cmu.edu.
4  Escape character is '^]'.
5  GET /cgi-bin/adder?15000&213 HTTP/1.0  Client: request line
6                                         Client: empty line terminates headers
7  HTTP/1.0 200 OK                        Server: response line
8  Server: Tiny Web Server                Server: identify server
9  Content-length: 115                    Adder: expect 115 bytes in response body
10 Content-type: text/html                 Adder: expect HTML in response body
11                                         Adder: empty line terminates headers
12 Welcome to add.com: THE Internet addition portal. Adder: first HTML line
13 <p>The answer is: 15000 + 213 = 15213  Adder: second HTML line in response body
14 <p>Thanks for visiting!                 Adder: third HTML line in response body
15 Connection closed by foreign host.     Server: closes connection
16 linux>                                Client: closes connection and terminates

```

图 11-28 一个提供动态 HTML 内容的 HTTP 事务

旁注 将 HTTP POST 请求中的参数传递给 CGI 程序

对于 POST 请求，子进程也需要重定向标准输入到已连接描述符。然后，CGI 程序会从标准输入中读取请求主体中的参数。

 **练习题 11.5** 在 10.11 节中，我们警告过你关于在网络应用中使用 C 标准 I/O 函数的危险。然而，图 11-27 中的 CGI 程序却能没有任何问题地使用标准 I/O。为什么呢？

11.6 综合：TINY Web 服务器

我们通过开发一个虽小但功能齐全的称为 TINY 的 Web 服务器来结束对网络编程的讨论。TINY 是一个有趣的程序。在短短 250 行代码中，它结合了许多我们已经学习到的思想，例如进程控制、Unix I/O、套接字接口和 HTTP。虽然它缺乏一个实际服务器所具备的功能性、健壮性和安全性，但是它足够用来为实际的 Web 浏览器提供静态和动态的内容。我们鼓励你研究它，并且自己实现它。将一个实际的浏览器指向你自己的服务器，看着它显示一个复杂的带有文本和图片的 Web 页面，真是非常令人兴奋（甚至对我们这些作者来说，也是如此！）。

1. TINY 的主程序

图 11-29 展示了 TINY 的主程序。TINY 是一个迭代服务器，监听在命令行中传递来的端口上的连接请求。在通过调用 `open_listenfd` 函数打开一个监听套接字以后，TINY 执行典型的无限服务器循环，不断地接受连接请求（第 32 行），执行事务（第 36 行），并关闭连接的它那一端（第 37 行）。

2. doit 函数

图 11-30 中的 `doit` 函数处理一个 HTTP 事务。首先，我们读和解析请求行（第 11~14 行）。注意，我们使用图 11-8 中的 `rio_readlineb` 函数读取请求行。

TINY 只支持 GET 方法。如果客户端请求其他方法（比如 POST），我们发送给它一个错误信息，并返回到主程序（第 15~19 行），主程序随后关闭连接并等待下一个连接请求。否则，我们读并且（像我们将要看到的那样）忽略任何请求报头（第 20 行）。